



University of Glasgow

CSC3106 Cyber Security

Data-Driven Security Analysis and Defensive Response

Team X

Name	GUID	SIT ID	Task Distribution
William Oon Zhen Wei	3070657O	2401532	Equal
Kieran Sim	3070564S	2403348	Equal
Tan Bing Kun Terence	3070672T	2401883	Equal
Dui Ru-En Joshua	3070683D	2402201	Equal

Part 1: Data-Driven Authentication Log Analysis

Assigned Extract: 4_auth.log
Chosen Asset: backup01 SSH access surface and privileged backup account
Analysis Script: analysis.py

1. Security Question and Scope

Security question. What phases of a cyberattack directed at `backup01` can be reconstructed through SSH authentication and session data within `4_auth.log`, and which defensive measures are most critical for securing privileged accounts?

The scope focuses on host `backup01`, centering on its SSH service and the operational `backup` user. This account was subjected to intense targeting and demonstrated an accepted login during an attack wave, followed by immediate `sudo` activity as `root`. The log period is Jul 06 to Jul 12. Quantitative data and visualisations were derived using `analysis.py`. The report reconstructs visible attack stages, separates empirical log data from interpretation, and highlights visibility gaps: `auth.log` cannot prove post-compromise actions such as data exfiltration or system tampering.

2. Method Summary

`analysis.py` decomposes log entries into syslog components before categorising `sshd`, PAM, `sudo`, and CRON messages into security events. It extracts user identities, source IPs, ports, commands, and authentication outcomes, identifies session transitions and privileged command execution, and preserves raw lines for review. Because the file lacks strict chronological ordering, timelines are reconstructed from parsed timestamps. Ordering anomalies are retained as investigation limitations, not treated as proof of tampering.

3. Evidence Overview

The extract contains 12,000 lines, all for `backup01`. Headline counts are shown in Table 1.

Table 1: Headline counts from `4_auth.log`.

Event type	Count	Event type	Count
Failed password	5,176	Session opened	1,047
Accepted password	1,859	Session closed	1,046
Invalid user	751	Max auth attempts	640
Preauth closed	752	Sudo command	255
CRON session	394	Possible break-in warning	80

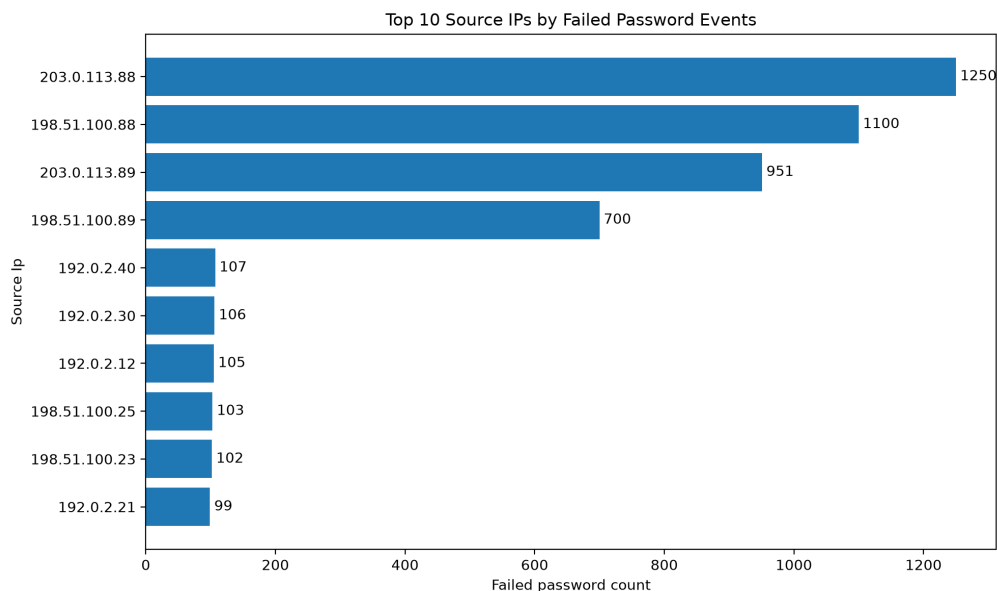


Figure 1: Top source IPs by failed password attempts. Four sources dominate SSH failure volume; 203.0.113.89 is notable because it later records a successful **backup** login.

The high failed-password volume indicates broad SSH credential guessing. Four sources dominate: 203.0.113.88, 198.51.100.88, 203.0.113.89, and 198.51.100.89. The username targeting (**root**, **admin**, **deploy**, **postgres**, **oracle**) and sustained volume are consistent with automated opportunistic SSH scanning rather than a bespoke campaign. The volume is therefore context, not proof of compromise. What elevates the finding is that 203.0.113.89 progresses from failures to a successful privileged-account login.

4. Attacker Lifecycle: The 203.0.113.89 Sequence

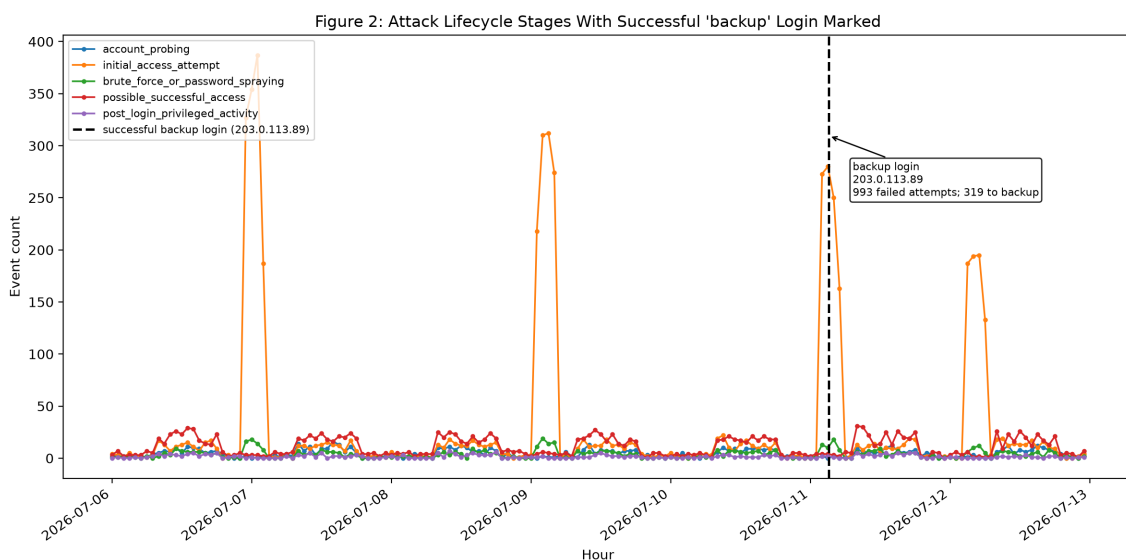


Figure 2: Attack lifecycle stages over time with the successful **backup** login marked. The successful login occurs during a major initial-access spike on Jul 11.

The strongest finding is a single-source lifecycle on Jul 11. 203.0.113.89 produced sustained failed attempts and probing, then a successful **backup** login, an SSH session, and immediate root-level sudo activity.

Table 2: Critical Jul 11 authentication-to-sudo evidence chain.

Time	Evidence	Lifecycle stage	ATT&CK mapping
03:07:30	Failed login for invalid user <code>oracle</code> from 203.0.113.89.	Account probing	T1110.001; consistent with T1087
03:07:32	Failed password for <code>root</code> from 203.0.113.89.	Privileged-account targeting	T1110.001
03:08:19	Accepted password for <code>backup</code> from 203.0.113.89.	Valid-account access over SSH	T1078; T1021.004
03:08:19	<code>pam_unix(sshd:session)</code> opened for user <code>backup</code> .	Session establishment	T1021.004
03:08:19	<code>backup</code> runs <code>journalctl -n 200</code> and <code>systemctl status cron</code> as root.	Privileged post-login activity	T1548.003

The sequence maps most defensibly to T1110.001 → T1078 → T1021.004 → T1548.003: password guessing reaches a valid account over SSH, whose sudo rights expose a path to root. The two sudo commands (`journalctl`, `systemctl status cron`) are individually legitimate administrative actions, so they are interpreted as possible privileged reconnaissance immediately after a suspicious login, not confirmed malicious action. Same-second timestamps reflect syslog’s one-second precision and do not prove millisecond ordering, but the rapid login-to-sudo transition is more consistent with scripted or prepared activity than routine manual administration.

Temporal pattern. Brute-force activity is concentrated outside working hours: roughly 70% of failed logins (3,643 of 5,176) fall between 00:00 and 05:59, with a further spike around 23:00. Legitimate accepted logins cluster in business hours (08:00–18:00). In the overnight window there are 3,643 failed against only 149 accepted logins; daytime is close to balanced (1,533 failed vs 1,710 accepted). The successful `backup` login at 03:08:19 therefore occurs inside the peak attack window and outside the host’s normal login baseline. Host-local time is assumed because no timezone is recorded.

5. Risk Prioritisation

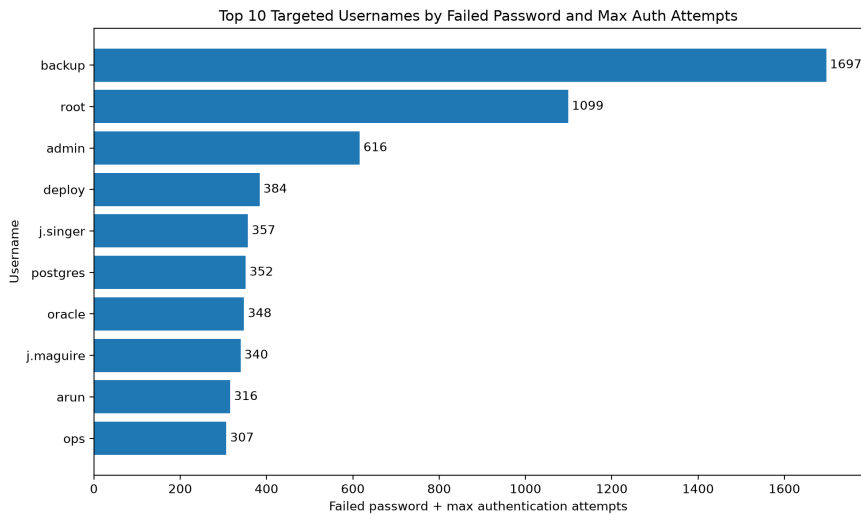


Figure 3: Top targeted usernames by failed-password plus maximum-authentication events. `backup` is the most targeted account, supporting its selection as the priority privileged asset.

Asset: `backup01`—its SSH access surface, the `backup` account, and the sudo-to-root path. The three prioritised risks trace the observed attack path: access, escalation, and impact.

Table 3: Prioritised risks for `backup01`.

ID	Risk	Likelihood	Impact	Evidence and uncertainty
R1	Unauthorised privileged SSH access via compromised <code>backup</code> credentials	High	High	951 failed passwords from 203.0.113.89, followed by accepted <code>backup</code> login. Cannot prove whether the login was authorised without external context.
R2	Escalation to root through <code>backup</code> sudo rights	High	Critical	<code>backup</code> runs <code>sudo</code> as root immediately after login. No destructive root command is visible in <code>auth.log</code> .
R3	Loss of backup-data integrity or availability	Medium	Critical	A root path exists on a backup host. <code>Auth.log</code> does not show backup-file access, exfiltration, or tampering.

R1 is the top priority: it combines attempted access, apparent success, and privileged activity in one source. Secondary risks—SSH-service degradation from high-volume brute force, and exposure of other sudo-capable accounts (`ops`, `arun`, `j.singer`, `j.maguire`)—are noted but not prioritised here. The targeting distribution also supports prioritising the `backup` account. As shown in Figure 3, `backup` received the highest number of failed-password and maximum-authentication events among extracted usernames.

6. Initial Response and Investigation Limitations

Immediate response should verify whether the `backup` login from 203.0.113.89 was authorised, rotate or disable the `backup` credentials until confirmed, preserve relevant logs, and review `sudo`, `cron`, `journal`, `shell` history, and audit evidence. High-volume SSH sources should also be blocked or rate-limited.

Table 4: Examples of timestamp-order anomalies. These are evidence limitations, not proof of tampering.

Line	Previous time	Current time	Interpretation
13	2026-07-06 00:49:51	2026-07-06 00:04:24	Session-close entry appears after later events in file order.
41	2026-07-06 02:19:09	2026-07-06 01:21:46	Earlier session-close line appears later in the extract.
70	2026-07-06 03:40:05	2026-07-06 02:56:33	File ordering is not strictly chronological.

This evidence is limited to `auth.log`. It shows authentication, session, and `sudo` activity, but not every shell command. Therefore, the extract supports likely brute-force activity, possible successful credential compromise, and privileged reconnaissance, but it does not prove botnet activity, malware execution, persistence, exfiltration, log deletion, or destructive tampering. The IP ranges are documentation-style ranges, so no real-world attribution or geolocation should be inferred.

Part 2: Technical Defensive Response

Selected risk. R1—unauthorised privileged access to `backup01` via the compromised `backup` account, reached through opportunistic SSH brute force from 203.0.113.89 and extended to root through the account’s `sudo` rights.

Defensive goal. The defence targets the full path rather than brute force alone:

SSH brute-force attempts → successful `backup` login → active session → `sudo` as root.

The response is layered: prevention to stop guessing succeeding, containment to limit damage if a credential is still compromised, and detection to catch a success the preventive layer misses.

Table 5: Recommended controls mapped to observed stages.

Attack stage	Control	Effect
Password guessing	Key-only SSH; <code>PasswordAuthentication no</code> ; rate-limit/fail2ban; <code>PermitRootLogin no</code>	Password guessing no longer grants access by default; brute-force noise is reduced.
Valid-account SSH access	Restrict <code>backup</code> key by source and forced command; disable interactive shell if not required	A stolen password is useless; even a stolen key only runs approved backup operations.
Privilege escalation via sudo	Least-privilege sudoers for <code>backup</code> ; no shell, <code>su</code> , or general sudo	A compromised <code>backup</code> context cannot obtain a general root shell.
Post-compromise tampering	SELinux enforcing, confining <code>backup</code> domain to required files	Limits writes to critical system files and persistence locations.
Evidence gaps after login	Central log forwarding plus sudo/audit logging	Preserves evidence beyond local <code>auth.log</code> .

Prevention and containment. Disabling password authentication removes the exposed password surface. Least-privilege sudo prevents `backup` from becoming a general root shell. SELinux in enforcing mode adds mandatory access control: the backup service should run in a confined domain permitted to touch only required backup paths. Even with residual privilege, access to sensitive paths such as `/etc/passwd`, `/etc/shadow`, `/etc/sudoers`, cron directories, SSH host keys, and logs can be denied by policy.

Detection. A naive “failed then accepted from the same IP” rule is too broad: legitimate users may mistype passwords before authenticating. Selectivity should come from volume, ratio, account sensitivity, timing, and sudo follow-on. Alert when a single source IP exceeds a failed-attempt threshold above the legitimate baseline, has a heavily skewed failure-to-success ratio, then obtains an accepted login to `backup`, especially outside business hours and followed by sudo. Applied to this extract, the rule flags 203.0.113.89.

Effectiveness and residual risk. Key-only SSH and root-login denial reduce likelihood by removing the password-guessing success path. Least-privilege sudo and SELinux reduce impact by preventing a compromised `backup` credential from becoming broad root control. Detection thresholds must be tuned: too low recreates false positives; too high may miss low-and-slow attackers. Rate limiting can disrupt operations if aggressive, key-only SSH shifts risk to key management, and SELinux requires careful policy maintenance. These controls improve prevention, containment, detection, and investigation, but they do not retrospectively prove what happened after the Jul 11 login.